NEXUS Language Reference

Version 2.0.0
Author: fanxcz
Repository: https://github.com/fanxcz/nexus-lang

---

1. Overview

NEXUS is a compiled programming language with two backends:

- **LLVM backend** — full language support, used for all serious programs.
- **Native backend** — small subset, useful for tiny programs and experiments.

The compiler is written in C++20. The runtime is written in C++. The standard library and a self-hosting toolchain are written in NEXUS itself.

A NEXUS program is a set of declarations. The entry point is fn main().

```
fn main() {
    print("Hello, NEXUS!")
}
```

Run it:

```
nexus run hello.nx --backend llvm
```

---

2. Installation and build

2.1 Build from source

```
cmake -S . -B build -G Ninja
cmake --build build -j
ctest --test-dir build
```

The compiler binary is build/nexus (or build/nexus.exe on Windows).

2.2 Environment

For self-hosting tests and .ll compilation, set NEXUS_CXX to a clang-compatible driver:

```
export NEXUS_CXX="/path/to/clang++"
```

On Windows the project provides build/clang++.cmd, which wraps the bundled Zig toolchain.

2.3 Project layout

```
src/        C++ compiler and runtime
stdlib/     Standard library written in NEXUS
selfhost/   Lexer, parser, type checker, code gener
docs/       Documentation
examples/   Example programs
tests/      Compiler and runtime tests
```

---

3. Quick start

Create add.nx:

```
fn add(a: i64, b: i64) -> i64 {
    return a + b
```

```
}

fn main() {
    print(add(40, 2))
    let x = add(add(1, 2), 3)
    print(x)
}
```

Run:

```
nexus run add.nx --backend llvm
```

Expected output:

```
42
6
```

---

4. Lexical structure

4.1 Comments

Line comment:

```
// this is a comment
```

Block comment:

```
/* this is
   a block comment */
```

4.2 Identifiers

Identifiers consist of letters, digits, and underscores. They must not start with a digit.

4.3 Keywords

```
fn let const mut return if else while for in break
struct enum import extern unsafe as match true fals
```

4.4 Literals

Integer:

```
42
-7
```

Floating point:

```
3.14
1.0
```

String:

```
"hello"
"line one\nline two"
```

Boolean:

```
true
false
```

4.5 Operators

```
+  -  *  /  %  ==  ≠  <  ≤  >  ≥  &&  ||  !  =
```

---

5. Types

NEXUS supports the following basic types:

Type	Description
i64	signed 64-bit integer
f64	64-bit floating point
bool	boolean
string	text
void	no value

Type annotations are used in function parameters and return types:

```
fn scale(x: f64, factor: f64) -> f64 {
    return x * factor
}
```

---

6. Variables

6.1 Immutable variables

```
let x = 10
```

6.2 Mutable variables

```
let mut i = 0
i = i + 1
```

6.3 Discard

```
let _ = some_function()
```

6.4 Type stability

Assignment must preserve the variable type:

```
let x = 10
x = 20      // ok
x = "text"  // error
```

---

7. Expressions

7.1 Arithmetic

```
a + b
a - b
a * b
a / b
a % b
```

Integer division is signed. Floating point division is used when either operand is f64.

7.2 Comparisons

```
a == b
a ≠ b
a < b
a ≤ b
a > b
a ≥ b
```

7.3 Logical operators

```
a && b
a || b
!a
```

7.4 Unary minus

```
-x
```

7.5 String concatenation

```
let s = "hello" + " " + "world"
```

7.6 Operator precedence

From highest to lowest:

1. function calls and primary expressions
2. unary -, !
3. *, /, %
4. +, -
5. comparisons
6. &&
7. ||

---

8. Control flow

8.1 If statement

```
if x > 0 {
    print("positive")
} else {
    print("not positive")
}
```

8.2 While loop

```
let mut i = 0
while i < 5 {
    print(i)
```

```
    i = i + 1
}
```

8.3 Blocks

Blocks use braces:

```
{
    let x = 1
    print(x)
}
```

---

9. Functions

9.1 Declaration

```
fn name(param: type, ...) → type {
    ...
}
```

9.2 Parameters

Parameters are declared with name and type:

```
fn add(a: i64, b: i64) → i64 {
    return a + b
}
```

9.3 Return

```
return value
```

A function may end without an explicit return in some contexts; the compiler inserts a default return where supported.

9.4 Main

```
fn main() {
    print("Hello")
}
```

main returns i32 at the LLVM level.

9.5 Calls

```
let x = add(1, 2)
print(add(x, 3))
```

9.6 Recursion

Functions may call themselves:

```
fn factorial(n: i64) → i64 {
    if n ≤ 1 {
        return 1
    }
    return n * factorial(n - 1)
}
```

---

10. Strings

10.1 Literals

```
let s = "hello"
```

10.2 Escapes

```
"\n"   // newline
"\t"   // tab
"\r"   // carriage return
```

```
"\\"   // backslash
"\""   // quote
```

10.3 Concatenation

```
let s = "a" + "b"
```

10.4 Printing

```
print("hello")
```

---

11. Floating point

```
let x = 1.5
let y = 2.0
print(x + y)
print(x * y)
print(x / y)
```

Comparisons:

```
if x < y {
    print("less")
}
```

---

12. Modules and imports

Import a module:

```
import "io.nx"
```

The standard library is located in stdlib/.

---

13. Standard library

13.1 io

```
read_line(prompt: string) → string
read_i64(prompt: string) → i64
read_f64(prompt: string) → f64
```

13.2 fs

```
fs_read(path: string) → string
fs_write(path: string, data: string) → i64
fs_exists(path: string) → bool
```

13.3 string

```
len(s: string) → i64
byte_at(s: string, index: i64) → i64
byte_len(s: string) → i64
```

13.4 os

```
args_count() → i64
args_get(index: i64) → string
```

13.5 net

```
http_get_text(url: string) → string
```

13.6 json

```
json_field(json: string, key: string) → string
```

13.7 db

```
db_open(path: string) → i64
db_exec(db: i64, sql: string) → i64
db_close(db: i64)
```

13.8 crypto

```
hash_sha256(text: string) → string
```

13.9 memory

```
memory_alloc(bytes: i64) → i64
memory_free(ptr: i64)
```

---

14. Runtime functions

The compiler declares runtime functions such as:

```
nexus_print_i64(i64)
nexus_print_f64(double)
nexus_print_bool(i1)
nexus_print_str(ptr)
nexus_str_concat(ptr, ptr) → ptr
nexus_str_equal(ptr, ptr) → i1
nexus_file_read(ptr) → ptr
nexus_file_write(ptr, ptr) → i64
nexus_file_exists(ptr) → i1
nexus_env(ptr) → ptr
nexus_system(ptr) → i64
nexus_exit(i64)
```

These are available to generated LLVM IR.

---

15. Tooling

15.1 Commands

```
nexus run file.nx --backend llvm
nexus check file.nx
nexus build
nexus repl
nexus fmt file.nx
nexus self-test
nexus doctor
```

15.2 Backends

- llvm — full language support.
- native — small subset only.

Use --backend llvm for all real programs.

15.3 Formatter

```
nexus fmt file.nx
```

15.4 LSP

NEXUS includes an LSP server for editor integration.

---

16. Build system

NEXUS uses CMake:

```
cmake -S . -B build
cmake --build build
ctest --test-dir build
```

Targets include the compiler, tests, and runtime tests.

---

17. Native backend

The native backend supports only a tiny subset:

- basic arithmetic
- simple control flow
- minimal runtime calls

Use the LLVM backend for strings, functions, f64, and standard library features.

---

18. Self-hosting

The selfhost/ directory contains:

- lexer_lib.nx
- parser_lib.nx
- typecheck_lib.nx
- codegen_lib.nx

The self-hosted code generator supports:

- let and let mut
- assignment
- print
- arithmetic
- comparisons
- logical operators
- unary minus
- if / else
- while
- functions and calls
- string literals
- f64

Self-hosting is declared complete only when the self-hosted compiler reproduces the C++ compiler output on the test corpus.

---

19. Grammar

Simplified grammar:

```
program     := declaration*
declaration := function | statement
function    := "fn" IDENT "(" params ")" ("→" type
params      := (param ("," param)*)?
param       := IDENT ":" type
type        := "i64" | "f64" | "bool" | "string" |
block       := "{" statement* "}"
statement   := let | assign | print | if | while |
let         := "let" "mut"? IDENT "=" expression
assign      := IDENT "=" expression
print       := "print" "(" expression ")"
if          := "if" expression block ("else" block)
while       := "while" expression block
return      := "return" expression?
```

---

20. Examples

20.1 Factorial

```
fn factorial(n: i64) → i64 {
    if n ≤ 1 {
        return 1
    }
    return n * factorial(n - 1)
}

fn main() {
    print(factorial(10))
}
```

20.2 Floating point

```
fn main() {
    let x = 1.5
    let y = 2.0
    print(x + y)
    print(x * y)
}
```

20.3 Strings

```
fn main() {
    print("hello")
    print("a" + " " + "b")
}
```

20.4 Loop

```
fn main() {
    let mut i = 0
    let mut sum = 0
    while i < 5 {
        sum = sum + i
        i = i + 1
    }
    print(sum)
}
```

---

21. Troubleshooting

Program does not run

Use the LLVM backend:

```
nexus run file.nx --backend llvm
```

Link errors

Ensure NEXUS_CXX points to a clang-compatible driver.

Type errors

Run:

```
nexus check file.nx
```

Self-hosting tests fail

Ensure a clang-compatible compiler is available and NEXUS_CXX is set.

---

22. Notes

- Functions must be declared before use.
- Use let mut for mutable variables.
- The native backend is limited.
- For full language support, use --backend llvm.